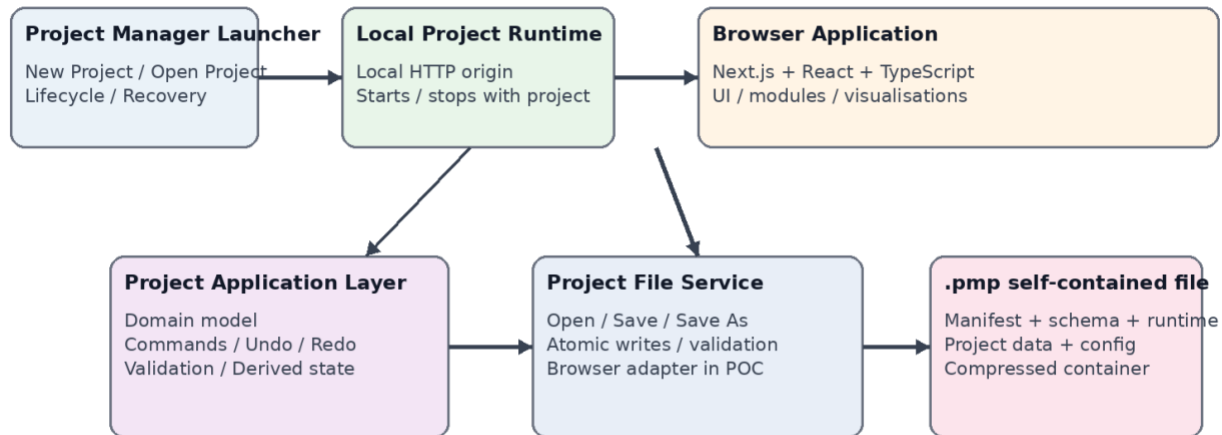


PROJECT MANAGEMENT APPLICATION

POC Requirements & Technical Specification

Codex-ready baseline specification

POC architecture and runtime boundary



Principle: the browser app is the product; Launcher and runtime are convenience/infrastructure boundaries.

Status: Baseline / implementation authority

Purpose A self-contained local project-management application delivered first as a zero-install browser POC, with a future desktop shell and portable .pmp project files.

1. Executive Summary

The product is a local-first project-management application for structured planning and execution. The POC runs without installation in a desktop browser, uses a local runtime, and persists each project as a single portable .pmp file. The product is deliberately not a generic task list: it is a connected project system in which streams, tasks, milestones, people, risks, decisions and relationships are shared across modules and visualised consistently.

The design is intentionally future-ready without implementing future complexity. The POC is offline, single-user, non-collaborative and free of cloud dependencies. The core domain and Project File Service are isolated from the browser so a future installed desktop shell (for example Tauri) can replace the POC launcher/file adapter without redesigning the project model or .pmp format.

Implementation authority Where implementation details are not explicitly specified, Codex should follow the architecture principles, safety boundaries and behavioural requirements in this document rather than inventing new product functionality.

2. Product Goals and Non-Goals

Goal	Definition
Fast local project management	Open/create a project and immediately work in a browser without installation or Internet connectivity.
One source of truth	All modules operate on the same project domain model and relationships.
Portable projects	A single .pmp file contains the project and the components required to recreate the project runtime within supported compatibility rules.
Operational clarity	Overview, Gantt and Tube Map provide complementary views of the same data.
Safe persistence	Autosave, Save Now, Save As, recovery and atomic file handling prevent silent data loss.
Future-ready architecture	Project File Service and domain isolation enable a future desktop shell.

Explicit non-goals

- No authentication, accounts, collaboration or multi-user editing.
- No cloud services, external APIs, telemetry or Internet dependency.
- No Jira, Azure DevOps, MS Project, Slack, Teams, GitHub or other integrations in the POC.
- No import/export, printing/PDF, attachments, comments, notifications or reminders.
- No resource capacity, utilisation, budget, cost, time tracking or baseline management.
- No mobile/tablet UI.
- No configurable custom fields, statuses, workflows or plugin marketplace.

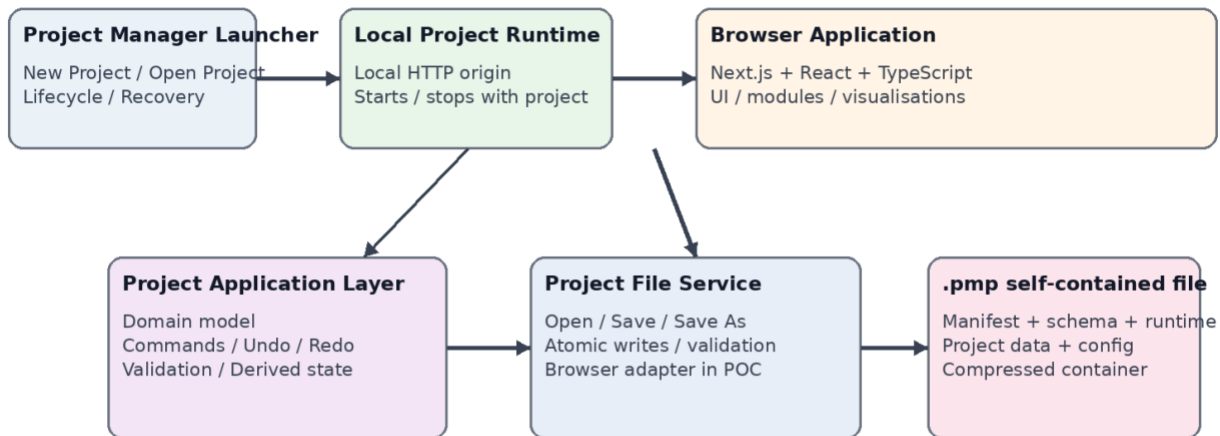
3. Scope and Operating Model

Area	POC decision
Deployment	Zero-install local browser application.
Browser	Chrome Desktop is official supported/tested environment; Chromium-compatible implementation where practical.
Operating systems	Windows and macOS.
Connectivity	Fully offline.
Concurrency	One project open at a time; same project cannot be concurrently edited by multiple instances.
Storage	Single .pmp file per project; browser storage is not project

Area	POC decision
	persistence.
Project lifecycle	Launcher -> open/create -> edit -> autosave -> close -> launcher.
Project identity	Stable Project ID + File Instance ID stored in the .pmp.

4. High-Level Architecture

POC architecture and runtime boundary



Principle: the browser app is the product; Launcher and runtime are convenience/infrastructure boundaries.

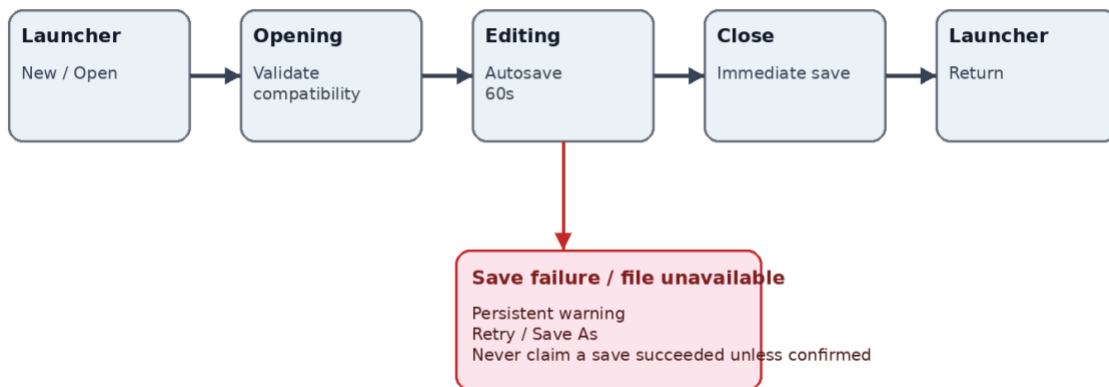
Figure 1. Architectural boundary. The browser application contains the product behaviour. The Launcher and local runtime are infrastructure boundaries. The Project File Service isolates project persistence from browser/native filesystem differences.

4.1 Architectural principles

1. Domain logic must not depend on React, browser APIs or UI components.
2. All project mutations occur through explicit application/domain commands or transactions.
3. Only one authoritative project state exists in memory.
4. KPIs, warnings, progress, matrices and visualisations are derived, not independently persisted.
5. The .pmp format is versioned and validated before project data enters the domain layer.
6. No arbitrary executable content may be loaded from a .pmp; embedded components are validated against the expected container contract.
7. The Project File Service provides the persistence abstraction required by both the browser POC and a future desktop adapter.

5. Runtime and Launcher Lifecycle

Project lifecycle and persistence states



Crash path: reopen from last successfully saved state; stale session marker can trigger recovery UI.

Figure 2. Normal lifecycle and failure path.

5.1 Launcher

- The Launcher is a separate, project-independent factory/convenience application.
- Its POC UI contains only New Project and Open Project.
- It does not maintain a project library, recent-project list, automatic reopening or a project registry.
- While a project is open, the Launcher remains available at the implementation level but may be hidden/minimised; the user experience must return it to the foreground when the project closes.
- If the project crashes, the Launcher detects the abnormal termination and provides Reopen Project / Open Another / Exit.
- After a second consecutive crash for the same project, the Launcher provides a stronger warning with Open Another / Exit and still permits Reopen Anyway.

5.2 Runtime

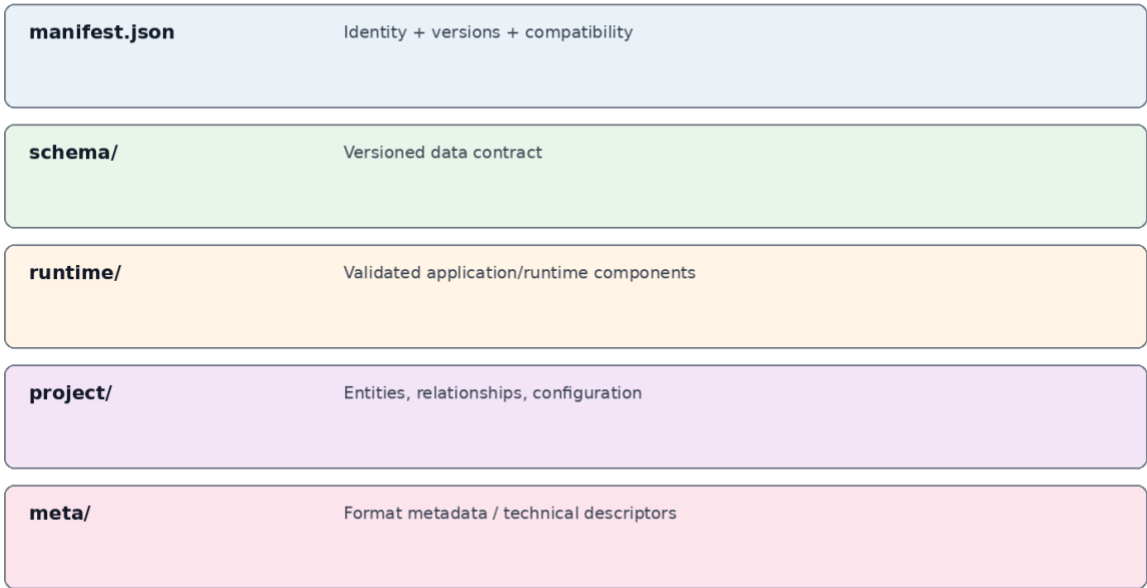
- The local runtime starts automatically for a project session and stops when the project session ends.
- Runtime technology is implementation-defined; the requirement is behavioural, not a mandate for Node, Python or another specific runtime.
- The runtime provides the local HTTP origin required by the browser POC.
- A direct browser development entry point may bypass the Launcher and must remain fully functional.

5.3 Crash and lock detection

- Use both runtime/process monitoring where available and a persisted session marker in OS-appropriate user-level application storage.
- The marker is never stored inside the .pmp file.
- A lightweight local session lock prevents concurrent editing of the same active .pmp.
- Stale locks are recovered automatically when the previous session is clearly gone; ambiguous state exposes Force Open with a warning.
- Moving/renaming a .pmp does not change its Project ID or File Instance ID.

6. .pmp Project File Specification

.pmp container anatomy



Compressed, self-contained, versioned; Project ID and File Instance ID travel with the file.

Figure 3. Logical .pmp container. The file is a compressed, versioned structured container. The exact low-level archive library is implementation-defined, but the logical contract must remain stable.

6.1 Mandatory identity and metadata

Field	Rule
Project ID	Stable UUID generated when project is created; travels with the .pmp.
File Instance ID	Separate UUID identifying the physical .pmp instance.
Format version	Mandatory version of the .pmp container contract.
Schema version	Mandatory version of project data schema.
Runtime compatibility	Embedded compatibility information required to determine whether the current runtime can open the file.
Project metadata	Name, dates, status and other project-level fields.

6.2 Logical contents

```
project.pmp
├── manifest.json
├── schema/
│   └── <versioned schema definitions>
├── runtime/
│   └── <validated application/runtime components>
├── project/
│   ├── entities/
│   ├── relationships/
│   └── configuration/
└── meta/
    └── <technical metadata>
```

6.3 File semantics

- The .pmp is the sole project persistence format and source of truth.
- No absolute filesystem paths are stored in project data.
- No external file references are supported in the POC.

- Browser LocalStorage/IndexedDB may hold ephemeral/session/UI state but never authoritative project data.
- Old files remain usable under compatibility rules; migration is explicit, never silent.
- Migration creates a new migrated .pmp and leaves the original untouched.
- If a file requires a newer runtime, opening is blocked with a clear explanation.

6.4 Identity and copying rules

- An externally moved/copied .pmp remains the same logical project unless the product explicitly gains a future Clone Project function.
- The POC has no Clone Project feature.
- File Instance ID is intended to distinguish physical file instances in the same local environment for session/crash detection.
- Across computers, the Project ID is portable and must not be silently changed.
- A filesystem move/rename is identity-neutral.

7. Persistence, Autosave and Save Safety

Requirement	Behaviour
Autosave cadence	Every 60 seconds.
Immediate persistence triggers	Project close/switch and application shutdown when changes exist since the last successful save.
Save Now	Ctrl/Cmd+S.
Save status	Compact header indicator: Saved · HH:MM. Show Saving... only if save exceeds approximately 500 ms.
Save failure	Persistent warning with Retry and Save Now; after repeated failure, offer Save As.
Save As	Default filename derives from Project Name; user can choose location/name. Successful Save As becomes active persistence target.
Overwrite	Existing target requires explicit Overwrite / Choose Another / Cancel confirmation.
Atomic save	Use temporary file + atomic replacement where supported; validate the generated container before replacement.
Unsaved state	Any .pmp-relevant mutation counts as unsaved, including project/module configuration changes and Undo after a save.
Crash	Reopen last successfully saved state only; no app-managed backup/recovery copies.

7.1 Persistence edge cases

- Network/shared storage is allowed with a warning that it is not the primary supported storage mode.
- If a network file disappears, keep the project open, show the critical file-access warning and offer Retry / Save As. Do not automatically overwrite when the file reappears.
- Read-only transition: offer Save As immediately and continue with the new file when successful.
- Disk full: show critical warning with Retry / Save As.
- If a file is deleted externally, continue editing with File Missing warning and offer Save As.
- If a new file appears at the same path, require an explicit Reload / Keep Current / Save As decision.
- If Save As fails, keep the project open with the existing unsaved warning and allow another Save As attempt.
- Closing with unresolved save failure permits Close Without Saving only through explicit destructive confirmation.

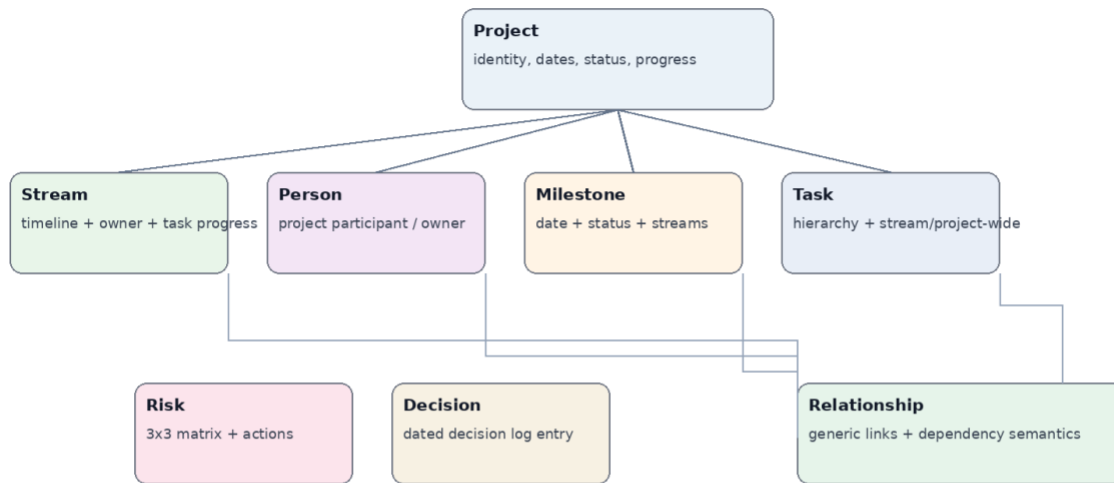
8. Undo / Redo

- Global session-scoped history across all modules, forms, inline edits, Gantt and Tube Map.
- Ctrl/Cmd+Z = Undo; Ctrl/Cmd+Y or Ctrl/Cmd+Shift+Z = Redo.

- History is not persisted in .pmp and is cleared when the project closes.
- Autosave does not create history entries.
- Delete and Restore are undoable; compound operations such as duplication are one atomic transaction.
- Successful Save As preserves history and establishes a new persisted baseline.

9. Domain Model

Core domain model and cross-module relationships



Membership uses project_id; generic relationships add explicit cross-entity semantics without duplicating membership.

Figure 4. Conceptual domain model. The final implementation may use a richer physical schema, but the semantics below are authoritative.

9.1 Common entity rules

- Every entity has a stable internal UUID and a human-readable project-local ID such as TASK-001.
- Human-readable IDs are sequential within a project; deleted IDs are never reused; duplicates receive next-highest ID + 1.
- Entities may optionally be associated directly with Project where the model defines Project-wide scope.
- Project membership is represented by project_id; generic relationships do not duplicate membership.
- Generic relationships are extensible; dependency is a specific relationship type with dependency semantics.

10. Project Entity

Field	Rule
Name	Mandatory.
Description	Optional.
Start Date	Mandatory.
End Date	Optional.
Status	Not Started / In Progress / On Hold / Completed / Cancelled.

Field	Rule
Progress	Manual 0–100 integer; independent of status.
Project ID	Stable UUID.
File Instance ID	Stable UUID for physical .pmp instance.

10.1 Project date semantics

- Project Start > Project End is critical and blocks the operation.
- Missing End Date is valid and causes timeline end to be derived dynamically when relevant dated content exists.
- Task/milestone/stream dates outside project dates generate warnings, not automatic corrections.
- Project Status never automatically changes because dates or progress change.

11. Streams

- Mandatory Name; optional Description, Owner, Start, End, Priority.
- Status uses the same fixed lifecycle as Project.
- No tasks => progress N/A.
- Progress is automatically calculated from leaf/flat tasks using equal weighting and nearest integer rounding.
- Parent tasks do not contribute separately to stream progress.
- Stream owner must be a project Person.
- Streams with no dates are excluded from the Overview timeline but appear in Stream Status Summary.

12. People

- Project-level entity; Name is initially the only mandatory field.
- People are not user accounts and have no authentication semantics.
- Person IDs are sequential and never reused.
- People can own multiple entities; each entity has zero or one owner in the POC.
- Deleting a person invokes reassignment flow; unassigned entities may retain the deleted person as historical owner with an inactive indicator.

13. Tasks

13.1 Task fields

Field	Rule
Name	Mandatory.
Description	Optional.
Scope	Exactly one Stream or explicitly Project-wide.
Status	Fixed lifecycle.
Start/End	Optional independently; start-only and date-less tasks are valid.
Owner	Optional Person.
Priority	Low / Medium / High.
Progress	Manual 0–100 for leaf/flat tasks; calculated for parent tasks.
Milestone	Optional association.
Dependencies	Optional; simple POC dependencies only.

13.2 Task hierarchy

- Flat tasks are valid.

- Nested hierarchy is optional and capped at four levels.
- Parent task progress = equal-weighted immediate children.
- Parent task status is derived from children.
- Parent start = earliest child start; parent end = latest child end; parent actual completion = latest child completion when all children are complete.
- Parent dates are calculated and not directly editable.

13.3 Task completion and overdue semantics

- Leaf/flat task completion records the current local calendar date as Actual Completion Date.
- Reopening clears Actual Completion Date; completing again records a new date.
- End date passed while status is not Completed => overdue indicator only; no automatic status change.
- Parent can leave Completed when children are reopened or newly added.

13.4 Dependencies

- POC supports simple prerequisite/dependent relationships across compatible entities and streams.
- Circular dependency chains are prevented.
- Future FS/SS/FF/SF semantics are architecturally extensible but not exposed in the POC.
- No automatic scheduling occurs.
- POC does not produce temporal warnings from dependencies.

14. Milestones

- Milestone is a distinct entity, not a zero-duration Task.
- Mandatory Name and Date; optional Description, Owner, Priority.
- Status = Tentative / Planned / Delayed / Complete.
- May belong to zero, one, multiple Streams, or explicitly Project-wide.
- May relate to multiple Tasks and dependencies.
- Completing a milestone records Actual Completion Date; reopening clears it.
- Past milestone date generates warning, not automatic status change.

15. Risks and Risk Log

Field	Rule
Title	Mandatory.
Description	Optional.
Status	Open / Mitigated / Closed / Accepted.
Probability	Low / Medium / High.
Impact	Low / Medium / High.
Risk Level	Derived from configured 3x3 matrix.
Owner	Optional Person.
Actions	Zero or more; Description mandatory; Status Open/Complete; Due Date optional.
Scope/Relations	Project-wide and/or related Streams and compatible entities.

15.1 Risk matrix

Probability	Impact	Risk Level
Low	Low	Low
Low	Medium	Low

Probability	Impact	Risk Level
Low	High	Medium
Medium	Low	Low
Medium	Medium	Medium
Medium	High	High
High	Low	Medium
High	Medium	High
High	High	High

- Risk status is manual.
- Accepted remains active for reporting; Closed is excluded from active counts.
- Actions may be overdue when due date has passed and status is Open.
- Completing all actions does not automatically close a risk.
- Reopening a risk preserves completed historical actions; new mitigation actions may be added.

16. Decision Log

- Decisions are lightweight records with mandatory Title and Date; optional Description and Owner.
- Can relate to Streams, Tasks, Milestones and Risks.
- No status and no separate rationale field in the POC.
- Deletion is soft deletion; restore returns relationships.

17. Soft Deletion and Restore

- Streams, Tasks, Milestones, People, Risks and Decisions use soft deletion.
- Deleted entities are hidden from normal views and calculations; Show Deleted reveals them.
- Relations are retained but become inactive while an entity is deleted.
- Restore restores the entity and retained relationships atomically.
- IDs are never reused.
- Bulk deletion is limited initially to Tasks, Risks and Decisions.
- Bulk restore is not supported.
- Delete confirmation is required only when relationships/dependencies are affected; otherwise deletion is immediate.

18. Duplication Rules

Entity	Duplicate semantics
Task	Copy fields except Status -> Not Started and Progress -> 0. Preserve dates, parent, stream/project-wide scope and active milestone. Do not copy dependencies or child tasks.
Milestone	Copy fields except Status -> Tentative and clear Actual Completion Date. Preserve planned date and active stream associations. Do not copy dependencies.
Risk	Copy fields except Status -> Open. Duplicate Actions as Open, preserving due dates. Copy relationships to active entities only; exclude deleted targets.
Decision	Copy fields; reset Date to current date. Copy relationships to active entities only.

- Supported duplicate entities: Tasks, Milestones, Risks and Decisions.
- Name is suffixed with (Copy).
- New sequential human-readable ID is generated.
- Duplication is one atomic Undo/Redo transaction and marks the project unsaved.

19. Overview / Dashboard

19.1 Header and global indicators

- Project name, project status, start/end timeline, Edit Project and Project Warnings indicator.
- Project Warnings indicator opens a complete derived warning list; opening does not clear warnings.

19.2 KPIs

KPI	Definition / action
Streams	Active streams only: excludes Completed/Cancelled. Click opens Streams module with active highlighted.
Active Tasks	Not Started + In Progress + On Hold, leaf/flat only, includes Project-wide. Click opens Task Management with active filter persisted for session.
Upcoming Milestones	Next five by date, statuses Tentative/Planned/Delayed; excludes Complete.
Open Risks	Open + Mitigated + Accepted; excludes Closed. Click opens filtered Risk Log.
High Risks	Risk Level High + active statuses. Click opens filtered Risk Log.
Delayed Milestones	Shown only when at least one exists; click opens filtered Milestones.
Overdue Items	Tasks + Milestones + Risk Actions; click opens combined contextual panel.

19.3 Timeline

- Visualises Project, Streams and Milestones only; Tasks are not shown.
- Future and attention-relevant content is prominent; completed historical content is de-emphasised.
- Project-wide milestones appear above streams as independent markers.
- No Project End Date => timeline end derives from farthest relevant dated element; if no dated elements exist beyond Project Start, show an empty timeline explanation.

20. Gantt

- Purpose: detailed planning/execution.
- Visualises streams, tasks, milestones, dependencies, dates, duration, status and progress.
- Supports create/edit, move dates, duration changes, progress adjustments, milestone creation and dependency management.
- Hierarchy shown as Project -> Stream -> Tasks with independent project-level milestones.
- Timeline zooms Day / Week / Month / Quarter; default Week.
- Project-wide tasks appear in a dedicated collapsible section; project-wide milestones appear at project level.
- Unscheduled tasks remain visible in lists and are visually distinguished.

21. Tube Map

- Purpose: high-level roadmap/narrative.
- Lines represent Streams; stations represent Milestones; Tasks are excluded.
- Milestones can span multiple Streams and appear as one shared station.
- Project-wide milestones cross all stream lines.
- Only milestone dependencies are shown.
- Calendar axis uses ISO weeks with Monday start; exact milestone dates are proportionally positioned within the week.
- Interactive: create milestones, drag dates, move milestones between streams where valid, create/manage milestone dependencies, open milestone details.

- Default visible range: Monday of the project start week through Monday of project end week.

22. Risk Matrix UI

- Display a 3x3 probability-impact matrix alongside the Risk Log table.
- Clicking a matrix cell filters/highlights risks matching the cell.
- Selecting a risk highlights its cell.
- Changing probability or impact immediately recalculates Risk Level and matrix position.
- Filters include Status, Probability, Impact, Risk Level, Owner, Related Stream/Milestone/Task, Due Date/Overdue.

23. Project Warnings Engine

- Warnings are derived; not persisted.
- Recalculate immediately after relevant changes and on project load/open.
- Undo/Redo triggers recalculation.
- All POC warnings have the same severity level; critical validation failures are separate blocking conditions.
- Warnings are a flat list with sort/filter and direct access.

Entity	Warning rule
Project	Start > End is critical. Missing dates alone are not warnings.
Stream	Start > End; outside project; task before stream start; task after stream end.
Task	Start > End; outside project; outside stream.
Milestone	Outside project. Multi-stream milestone warns only if outside the period of all related streams. Project-wide milestones exempt.
Dependencies	No dependency temporal warnings in POC.

24. Navigation, Search and UI State

- Persistent left sidebar with Project Overview/Dashboard landing.
- Global search Ctrl/Cmd+K searches current and future entity types by name/title and human-readable ID.
- Search results are grouped by module/type and support Quick View followed by full detail.
- Deleted entities are excluded by default; Include Deleted reveals them with Deleted indicator, Quick View and Restore.
- Module filters and sort are remembered for the current session only.
- Cross-module navigation opens the owning module, preserves the target module state, and opens the detail panel.
- Closing a detail panel returns the user to the preserved module context.

25. Forms, Validation and Editing

- Detailed creation/full-edit forms use modal or side-panel forms; common fields support inline editing in context.
- Critical validation blocks completion; non-critical inconsistencies become warnings.
- Mandatory fields validate on create/complete; invalid forms remain open with highlighted fields.
- Warnings are applied immediately without confirmation.
- Closing an edited detail modal/panel applies changes (equivalent to Done).
- Esc closes modal/panel.
- Unsaved changes inside a Project Details form: Discard asks Keep Editing / Discard; no prompt if there are no changes.

26. Project Settings and Technical Information

Area	Contents
Project Details	Name, Description, Start Date, End Date, Status. Opened from Overview via Edit Project.
Project Settings	User-facing project configuration and display preferences.
File Information	Read-only technical metadata: format version, app/runtime version, schema version and file metadata.

- POC display preferences: Monday week start, DD/MM/YYYY display, comfortable density. Compact density is internally future-ready but hidden.
- Module visibility is internally configurable but all POC modules are visible.

27. Onboarding and Quick Actions

- New project opens directly to an empty Overview.
- An adaptive checklist contains exactly four steps: Project, Streams, People, Milestones.
- Checklist is data-driven and disappears automatically when all steps are complete.
- Quick Actions prioritise onboarding gaps first; afterward they show relevant standard actions without duplicating full module administration.
- Add Task on Overview appears only when at least one Stream exists.

28. Accessibility and Internationalisation

- Core workflows support keyboard navigation; forms, tables, modals and filters are operable without requiring a mouse where practical.
- Focus moves into opened dialogs and returns to the triggering control on close.
- Colour is never the sole indicator of status/severity.
- Semantic HTML, labels and accessible controls support screen readers; formal WCAG certification is outside POC scope.
- UI language is English in POC; all strings are structured for future localisation.
- User-entered content supports Unicode and any practical language supported by the browser/runtime.

29. Date and Time Semantics

- Date-only project fields are stored as date values, not hidden timestamps.
- ISO 8601 / machine-readable representations are used internally where appropriate.
- POC UI week start is Monday; Tube Map uses ISO week numbers.
- Current local calendar date is used for completion timestamps represented as date-only values.
- Timezone handling uses a timezone database rather than hard-coded UTC offsets; Europe/Madrid is the initial default context.

30. Security and Trust Model

- No outbound telemetry, analytics, update checks or network calls in normal POC operation.
- No passwords, API keys, tokens or other secrets are stored in .pmp.
- Runtime/debug logs remain outside project data.
- Opening a .pmp requires strict validation of container structure, manifest, schema, expected components and size/path safety.
- Malformed or malicious files fail closed; arbitrary embedded code is not accepted.
- Any repair or migration output is written to a new file and never mutates the original.

31. Performance and Scale Targets

The following are engineering target sizes, not hard business limits. The application should remain correct if exceeded, subject to hardware capability.

Entity	Target scale
Streams	500
Tasks	10,000
Milestones	2,000
People	2,000
Risks	5,000
Decisions	5,000

- Use virtualised/lazy rendering for large lists and Gantt/visual views where needed.
- Derived calculations may be incremental, but a reliable full recalculation must be available on project load.
- Performance degradation must never result in silent data loss or omission.
- No dedicated GPU or high-end workstation should be required for the POC target.

32. Testing Strategy and Definition of Done

Test layer	Required coverage
Unit	Domain rules, calculations, validation, ID generation, soft delete, duplication, migration utilities.
Integration	Project persistence, .pmp round-trip, Project File Service, recovery, external file changes, save failures.
UI journey	Create/open/edit/save/close, onboarding, search, module navigation, delete/restore, forms, filters.
Visual behaviour	Gantt/Tube Map underlying transformations and selected rendering states.
Security	Malformed containers, path traversal, oversized inputs, unexpected components, unsafe runtime content.
Cross-platform	Windows + macOS with Chrome Desktop.
Crash/recovery	Interrupted saves, stale locks, missing files, repeated crashes and recovery flows.

32.1 Definition of Done

- Feature implemented against this specification.
- Domain behaviour covered by automated tests.
- Persistence round-trip validated where applicable.
- Accessibility and error states implemented.
- No known regression in critical flows.
- Documentation updated where architecture or format changes.
- CI passes tests and quality/security checks.

33. Recommended Implementation Sequence

1. Project domain model and invariants.
2. .pmp container manifest, schema and round-trip persistence.
3. Project File Service abstraction + browser adapter.
4. Launcher/runtime lifecycle and session/lock handling.
5. Overview and project metadata editing.

6. Streams and People.
7. Tasks, hierarchy, progress and dependencies.
8. Milestones.
9. Gantt.
10. Tube Map.
11. Risk Log and risk matrix.
12. Decision Log.
13. Warnings engine.
14. Global search, cross-module navigation and UI-state persistence.
15. Undo/Redo across all supported mutations.
16. Failure/recovery hardening and edge cases.
17. Accessibility, performance and UI polish.
18. Full test suite, migration tests, security tests and documentation.

34. Acceptance Criteria — POC Release Gate

ID	Acceptance criterion
AC-01	Create project from Launcher creates a valid .pmp and immediately opens it.
AC-02	Open Project loads a compatible .pmp and restores the last successfully saved state.
AC-03	Autosave occurs every 60 seconds and on relevant close/switch events.
AC-04	Save failures are never reported as successful; Retry and Save As paths are available.
AC-05	Undo/Redo works globally across all supported entity mutations and preserves expected unsaved state.
AC-06	Project/Stream/Task/Milestone/Risk/Decision relationships remain consistent after save/reload.
AC-07	Soft delete excludes entities from normal counts and restore reactivates relationships.
AC-08	Duplications follow the field-copy/reset rules and are atomic in Undo/Redo.
AC-09	Overview KPIs and visualisations update immediately when underlying data changes.
AC-10	Gantt and Tube Map reflect the same underlying project state without duplicating business data.
AC-11	Malformed or unsupported .pmp files fail safely without modifying the original.
AC-12	Same-project concurrent editing is blocked; stale lock recovery is safe.
AC-13	Project moves/renames remain identity-neutral.
AC-14	POC operates fully offline on supported Windows/macOS Chrome Desktop environments.
AC-15	CI passes automated tests and the implementation documentation matches the released behaviour.

35. Implementation Notes for Codex

Primary instruction Implement the product as a coherent system rather than a collection of page mock-ups. Reuse one domain model, one command/mutation pathway, one persistence abstraction and derived views everywhere.

- Prefer small, composable modules and explicit interfaces over framework-coupled logic.
- Keep all business rules testable without a browser.
- Keep UI state separate from persistent project state.
- Do not add out-of-scope functionality unless required to satisfy an explicit technical dependency; document any unavoidable infrastructure addition.
- When two requirements appear to conflict, protect data integrity, explicit user control and the single-source-of-truth model first.
- When an unspecified detail is needed, choose the simplest implementation consistent with this specification and future desktop portability.

36. Future Roadmap — Explicitly Deferred

Future capability	POC status
Installed desktop application / Tauri shell	Deferred; architecture prepared.
Native filesystem adapter	Deferred; Project File Service abstraction prepared.
File association for .pmp	Deferred to desktop shell.
Explicit Clone Project	Deferred.
Encryption/password protection	Deferred.
Custom fields/statuses/workflows	Deferred.
Import/export and integrations	Deferred.
Collaboration/multi-user	Deferred.
Resource/capacity and financial management	Deferred.
Attachments/comments/notifications	Deferred.
Advanced dependency types and scheduling	Deferred.

37. Consolidated Decision Baseline

This specification consolidates the full decision process completed for the POC. The original decision sequence covered product scope, data model, UX, persistence, launcher/runtime lifecycle, file identity, crash recovery, security, performance, accessibility, testing and implementation architecture. Later grouped rounds intentionally replaced one-question-at-a-time interrogation with recommended defaults. All recommended defaults were accepted.

Decision area	Baseline
POC architecture	Zero-install local browser app with local runtime and Project File Service abstraction.
Project format	Versioned compressed .pmp container, self-contained and portable.
Persistence	60-second autosave + explicit save/recovery controls.
Concurrency	Single active editor per project/file.
Identity	Project ID + File Instance ID; filesystem move/rename neutral.
Data model	Connected project system with Streams, Tasks, Milestones, People, Risks, Decisions and generic relationships.
UX	Persistent left navigation, Overview landing, inline + detailed editing, Gantt and Tube Map.
Integrity	Soft delete, atomic restore/duplication, global Undo/Redo, derived warnings.
Security	Offline, no telemetry, strict .pmp validation, no arbitrary

Decision area	Baseline
	executable content.
Testing	Unit, integration, UI journeys, security, recovery, cross-platform and CI.

38. Appendix A — Keyboard Shortcuts

Shortcut	Action
Ctrl/Cmd + Z	Undo
Ctrl/Cmd + Y	Redo
Ctrl/Cmd + Shift + Z	Redo alternative
Ctrl/Cmd + S	Save Now
Ctrl/Cmd + K	Global Search
Esc	Close modal/panel

39. Appendix B — Module Responsibility Matrix

Module	Primary responsibility	Primary entities
Overview	Project narrative, KPIs, warnings, quick actions	Project, Streams, Tasks, Milestones, Risks
Streams	Stream definition/status	Streams
People	Project people/ownership	People
Tasks	Task planning and hierarchy	Tasks
Milestones	Milestone planning	Milestones
Gantt	Detailed schedule visual + editing	Streams, Tasks, Milestones, Dependencies
Tube Map	Roadmap narrative	Streams, Milestones, Milestone dependencies
Risk Log	Risk register + matrix	Risks, Risk Actions
Decision Log	Decision register	Decisions
Project Warnings	Derived date consistency warnings	Derived
Project Settings	User-facing project configuration	Project configuration
File Information	Technical metadata	File/runtime/schema metadata

40. Appendix C — Key Behavioural Invariants

- No project data is silently discarded.
- No save is reported as successful without confirmed persistence.
- No module owns a private copy of shared project data.
- No deleted entity is included in normal KPIs or derived calculations.
- No ID is reused after deletion.
- No dependency can introduce a circular chain.
- No unsupported .pmp version is opened as though it were compatible.
- No arbitrary runtime/executable content is accepted merely because it is present inside a .pmp.
- No Project Status automatically drives entity status or Project Progress.
- No future/advanced feature is accidentally implemented as a side effect of the POC architecture.